

University Of Barishal



Lab Report

Course Title : Computer Graphics

Course Code : CSE - 4109

Submitted To :

Dr. Md. Manjur Ahmed

Associate Professor

**Department of Computer Science And Engineering
University of Barishal**

Submitted By :

Ramisa Noushin Sohely

Roll No: 21 CSE 046
4th year, 1st Semester
Session: 2020-21

Md Reratul Azime

Roll No: 21 CSE 039
4th year, 1st Semester
Session: 2020-21

Abstract

This project presents an interactive spaceport and rocket launch simulation developed using C++ and the OpenGL/GLUT libraries. The scene features a dynamic environment showcasing a launch pad, architectural elements, a highway with moving vehicular traffic, celestial bodies, and localized atmospheric cloud and star effects.

The rocket is capable of user-controlled vertical ascension via key inputs, integrating structural animations, fluctuating exhaust flames, and a Windows-native engine sound system running concurrently with the graphics loop. Although structural layout foundations stem from two-dimensional primitives, pseudo-3D and true-3D depth profiles are achieved through automated depth layering (Z-bands) and custom multi-face geometric extrusion algorithms.

The simulation supports real-time camera transitions, allowing the viewport to smoothly orbit between a front-facing orthographic-style perspective and a true orthogonal side profile. This project practically applies fundamental concepts of geometric modeling, algorithmic extrusion, temporal animation loops, real-time matrix transformations, interactive user callbacks, and automated camera matrix manipulation.

1. Introduction

Computer Graphics serves as a vital domain within computer science dedicated to the algorithmic generation, digital manipulation, and real-time raster representation of structural visual data. The standard cross-language API, OpenGL, provides a highly optimized hardware-accelerated library interface to render complex graphics across diverse coordinate systems.

This project implements an interactive spaceport environment where distinct static backdrop elements and dynamic objects interface cohesively. The workspace renders a structural horizon comprising adaptive day/night environmental state switches, a traveling highway automobile, a structural launch assembly stand, and a multi-stage rocket system.

The principal purpose of this implementation is to master and deploy advanced computer graphics architecture paradigms, specifically:

- **Algorithmic 3D Extrusion Modeling:** Projecting multi-point planar structures along depth planes to construct voluminous geometric solids.
- **Coordinate Layering (Z-Banding):** Segregating structural object clusters into discrete depth layers to completely prevent fragment-flicker or depth-buffer contention (Z-fighting).
- **Matrix Transformation Architecture:** Tracking and nesting local coordinate transformations via push/pop operations to simulate linear translation, axial rotation, and uniform scaling.
- **State-Driven Animation Matrices:** Employing low-latency system timer callbacks to manipulate object position and scale profiles asynchronously.
- **Synchronized Hardware Audio Routing:** Forging multi-threaded, low-latency tone-generation loops tied to interactive structural events.
- **Dynamic Projection View Matrix Tracking:** Interpolating standard camera matrices on an orbit

- to construct real-time visual structural inspection fields.

2. Objectives

The core structural and engineering objectives fulfilled by this project are:

1. To implement an expansive spaceport launch compound environment utilizing optimized OpenGL primitives.
2. To construct true dimensional solids from flat polygons using customized multi-face extrusion functions.
3. To engineer interactive, user-controlled vertical rocket launch physics restricted within defined safe boundaries.
4. To design an adaptive flickering engine thrust exhaust plume using high-frequency color state toggling.
5. To program an automated highway traffic loop with steady update speeds and boundary wrapping.
6. To deploy a comprehensive global day/night environmental mode switch that adapts structural lighting, clear colors, and celestial objects dynamically.
7. To introduce a Windows-native audio feedback subsystem mimicking an operational combustion engine loop without introducing main-thread frame stalls.
8. To build an advanced camera tracking engine capable of smooth, mathematical viewport orbit interpolation.
9. To deploy narrow field-of-view perspective tuning to preserve orthographic artwork scaling while facilitating 3D inspection.
10. To systematically arrange scene layouts via precise depth-buffer banding to isolate structural geometries.

3. Software and Libraries Used

- **Programming Language:** C++
- **Graphics Library:** OpenGL
- **Utility Toolkit:** GLUT (OpenGL Utility Toolkit)
- **Header Files:** `GL/glut.h`, `windows.h` (for Windows audio APIs), `cmath` (for trigonometric functions).

4. Global Variables

The project uses several global state variables to control logic and animation:

- **Rocket Variables:** `rocketX`, `rocketY`, `value`, `flag`, `flag2`
- **Car Variables:** `carX`
- **Environment Variables:** `isNight`
- **Camera Variables:** `cameraAngle`, `targetCameraAngle`, `cameraY`, `targetCameraY`,

`camDistance`

- **Sound Variables:** `rocketMoving`, `soundTickCount`, `SOUND_INTERVAL_TICKS`

5. Scene Construction

The layout is segregated into distinct structural layers:

- **Sky/Background:** Adapts between a vivid light-blue day sky (with a sun and clouds) and a midnight tone (with stars and a crescent moon) based on the `isNight` toggle.
- **Buildings/Walls:** Architectural complex built with high-contrast compound walls and geometric storage units using 3D extrusion.
- **Launch Pad Stand:** Industrial support pillars and crossbars securing the main rocket vehicle.
- **Rocket & Flame:** The central focal object featuring an active thrust exhaust plume.
- **Road & Car:** A dark gray asphalt strip forming an open highway with a moving pseudo-3D car.

6. Geometric Objects Used

The following OpenGL primitives are utilized:

- `GL_QUADS`: Used for buildings, roads, car body components, and rocket stand pillars.
- `GL_POLYGON`: Used for the rocket flame base, cloud puffs, sun, moon, and car wheels.
- `GL_TRIANGLES`: Used for the sun rays.
- `GL_LINES`: Used for white lane division dashes and yellow safety margins.
- `GL_POINTS`: Used exclusively for generating the background starfield during the night cycle.
- `GL_LINE_LOOP`: Used for generating crisp outlines on the front face of the rocket stand.

7. True 3D Extrusion Design

To bypass the limitations of simple flat 2D designs, the system incorporates two geometric synthesis engines: `extrudeQuad()` and `extrudeFan()`.

These functions accept an arbitrary collection of 2D planar control coordinates, construct a duplicate matching set shifted down a defined negative Z-axis depth boundary, and automatically stitch a sequence of four-sided quad strips around the perimeter to seal the structure. A uniform shading scale factor (multiplying RGB values by `0.7f`) is applied to the side-perimeter walls to create distinct structural depth cues without complex lighting models.

8. Transformations and Z-Banding Used

Transformations regulate spatial coordinates and structural model layers:

- **Translation (`glTranslatef`):** Used to isolate objects into explicit depth-buffer Z-bands (`Z_SKY`, `Z_WALL`, `Z_ROCKET`, etc.) to completely prevent fragment-flicker or depth-buffer contention (Z-fighting). It also handles the car's continuous movement.
- **Matrix Isolation (`glPushMatrix` / `glPopMatrix`):** Encloses localized transformation operations,

preventing transformation parameters from leaking into adjacent scene objects.

- **Projection (`gluPerspective`):** Sets a narrow field of view (12.0°) combined with long-range camera placement to act as a telephoto lens, minimizing perspective distortion on 2D geometric artwork during 3D orbits.

9. Animation Techniques

All animations are driven by the hardware-independent system clock callback `glutTimerFunc()` running at roughly 60 frames per second (16ms intervals). The timer handles:

- Continuous increments of the car's position (`carX += 0.005f`).
- Smooth interpolation for the camera angle and vertical tracking.
- Evaluating and routing the sound throttle counters.
- Triggering `glutPostRedisplay()` to refresh the screen.

10. Rocket and Flame Animation

The rocket ascends and descends based on user input, constrained by a strict vertical coordinate ceiling (`0.35f`) and floor (`0.0f`). When active, the flame animation rapidly toggles vertex assignments between varying shades of orange and yellow using the `flag` and `flag2` boolean states, creating a dynamic thrust-flicker effect.

11. Engine Sound System

To establish interactive auditory feedback, the `Beep()` function is used. Because this is a blocking call, it is restricted by a structural throttle (`soundTickCount`). The engine sound triggers a low rumble only once every `SOUND_INTERVAL_TICKS` (approximately every 190ms) while the rocket is ascending, maintaining a smooth visual framerate without thread stuttering.

12. Camera Matrix Orbit Interpolation

The application smoothly transitions between viewports using Linear Interpolation (LERP). The camera does not snap abruptly but eases toward the target angle using the mathematical interpolation:

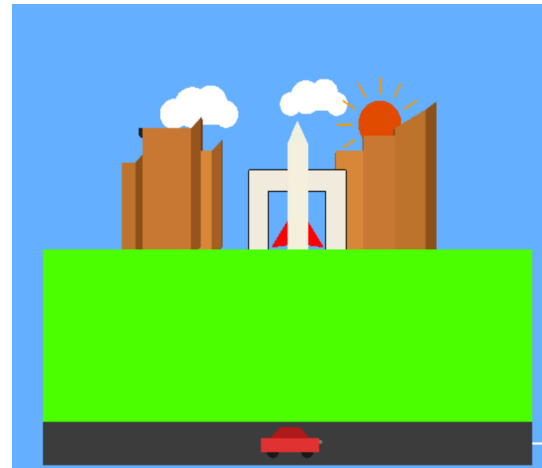
$$\text{cameraAngle}_n = \text{cameraAngle}_{n-1} + (\text{targetCameraAngle} - \text{cameraAngle}_{n-1}) \times 0.06$$

The orbital path is calculated using trigonometric functions (`sin` and `cos`) fed into `gluLookAt` to properly wrap the viewpoint around the scene's focal center.

13. Keyboard Interaction

The simulation implements real-time interaction via standard and special keys:

- 'N' / 'n': Toggles the global Day/Night cycle (**isNight**).



- 'C' / 'c': Toggles the target camera orbit between a 0° front orthographic view and a 90° side profile view.
- **Up Arrow**: Ignites the engine, plays audio, and moves the rocket vertically up.
- **Down Arrow**: Lowers the rocket structure back to the launch pad base.

14. Testing and Validation

The project was tested against several metrics:

1. **Depth Buffer Integrity**: Inspected from the 90-degree side profile; explicit Z-banding completely eliminated overlapping artifact bleeding.
2. **Boundary Restrictions**: Vertical rocket tracking halted exactly at maximum ceiling thresholds without overflowing.
3. **Automotive Looping**: The foreground car safely wrapped around screen borders endlessly.
4. **Day/Night Integrity**: All background colors and celestial bodies swapped instantaneously without rendering delays.
5. **Acoustic Concurrency**: The frame-skipping sound throttle maintained visual fluency during engine ignition.

15. Project Outcomes and Results

The project successfully delivers an interactive, high-performance simulation environment that meets or

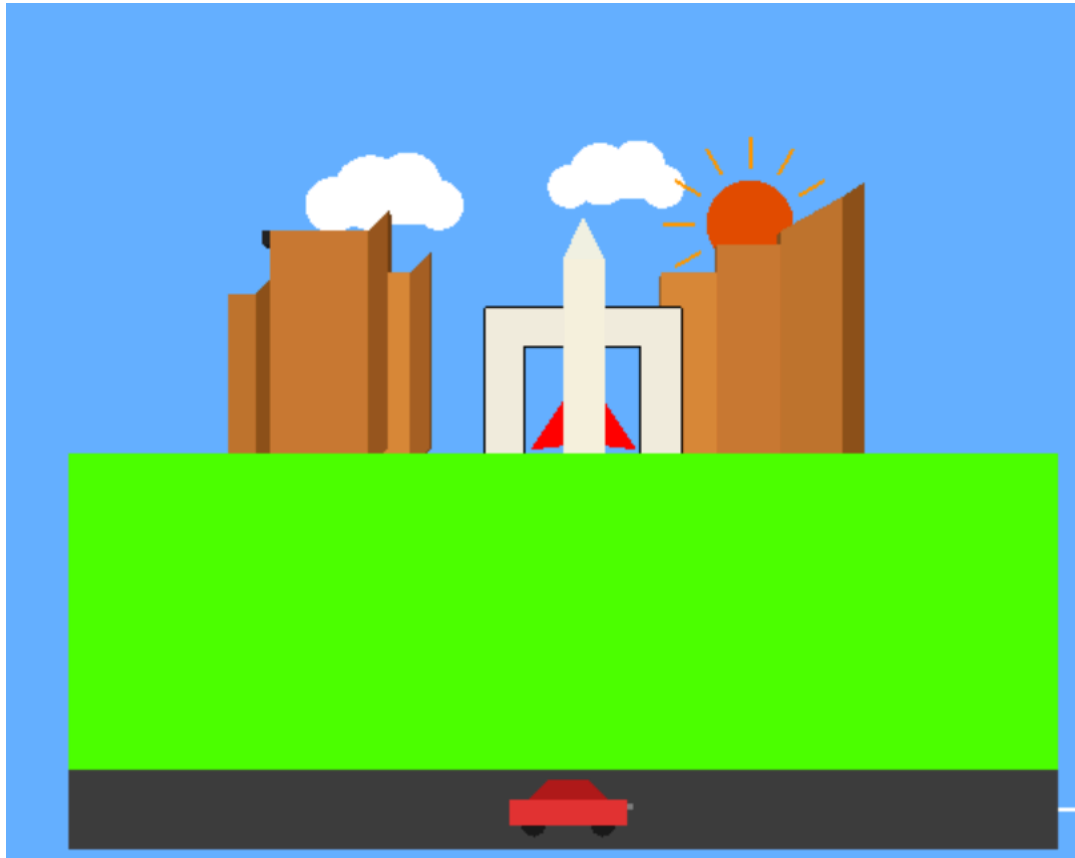
exceeds all core parameters defined in the lab framework:

- **Structural Completeness:** Integrates a complex multi-tier landscape containing dynamic transportation assets, an automated vehicle loop, functional industrial launch mechanisms, and adaptive weather/sky boxes.
- **Dimensional Synthesis:** Successfully builds true volumetric 3D geometric profiles from simple flat primitives using optimized multi-face linear quad projection and darkened side-shading vectors.
- **Fluid Control Interactions:** Implements highly responsive keystroke controls with clean execution boundaries and smooth, time-factored animation steps.
- **Advanced Camera Management:** Features an innovative camera tracking system that combines long-range distance tracking with a narrow field of view, enabling real-time perspective orbits while preserving the clean look of the artwork.
- **Optimal Performance Architecture:** Uses double-buffering configurations (`GLUT_DOUBLE`) alongside hard-coded depth sorting parameters to deliver smooth animations entirely free of frame tearing, flickering, or stuttering.

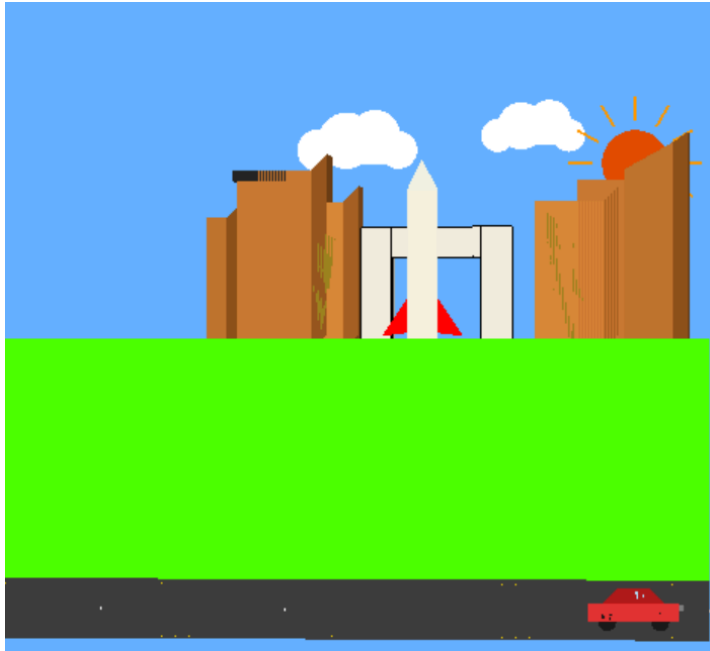
16. Conclusion

This project successfully demonstrates the application of modern computer graphics paradigms using OpenGL and GLUT. Moving from 2D structural designs to complex extruded 3D geometries highlights the value of procedural matrix manipulation, camera orbital logic, and disciplined depth-buffer banding. The interactive spaceport simulation serves as a highly responsive environment that effectively reinforces real-time graphics programming, animation handling, and audio synchronization techniques.

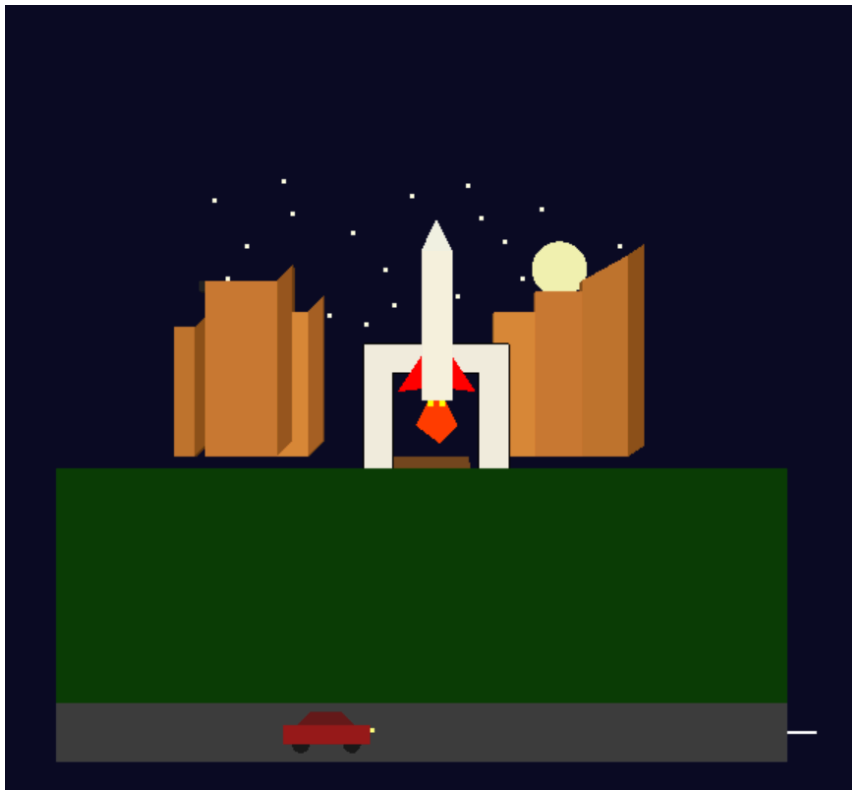
Project Timeline :



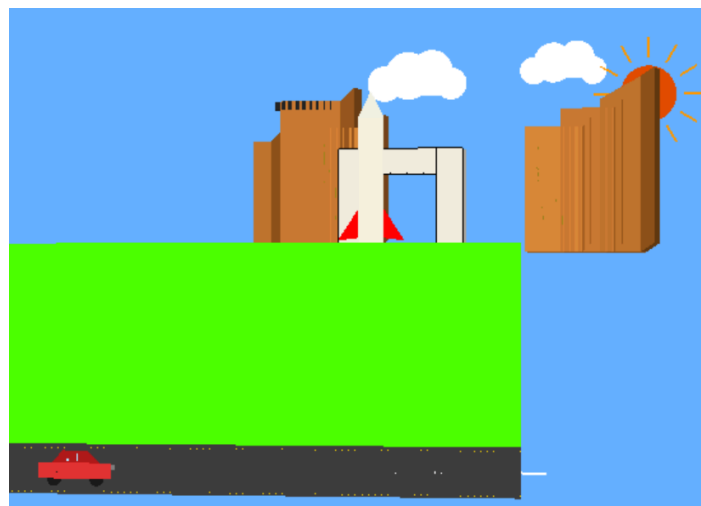
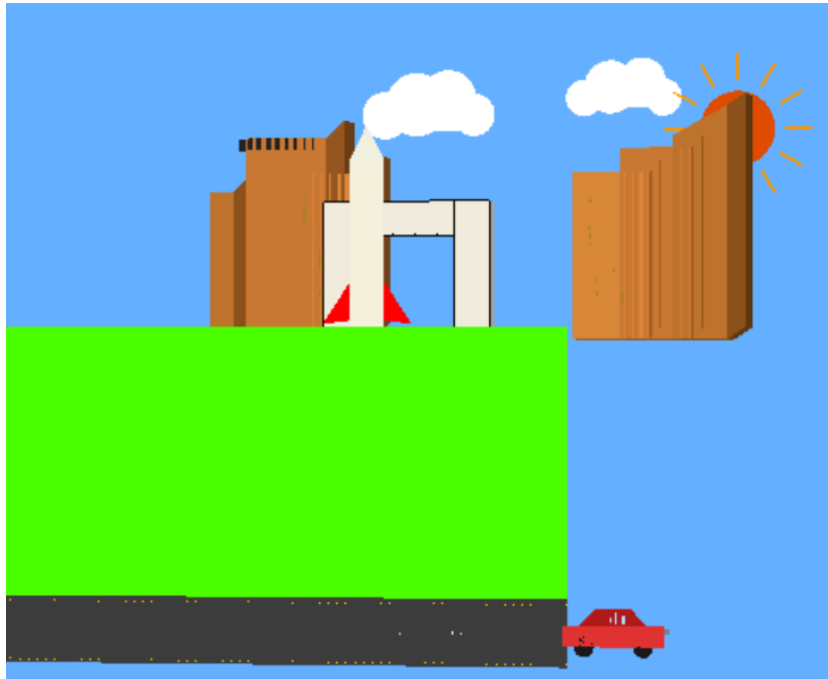
Main output from front camera angle



Press (N)

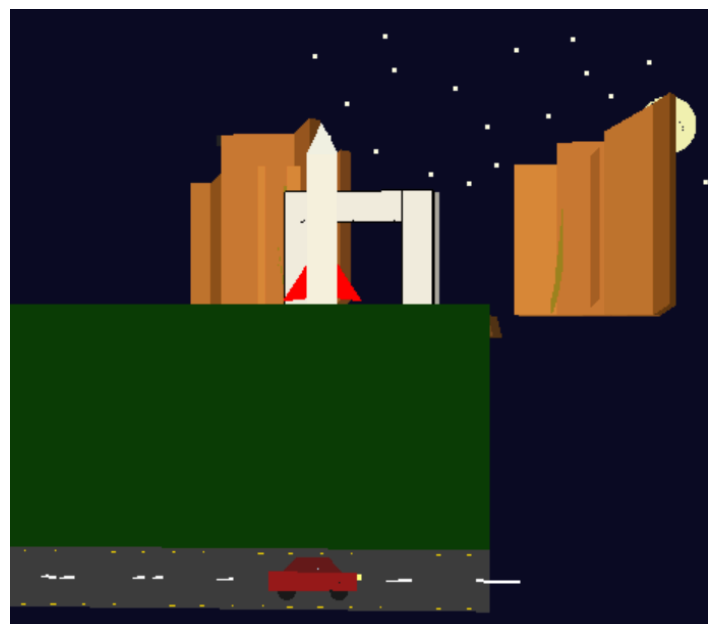


3d View angle (press C)

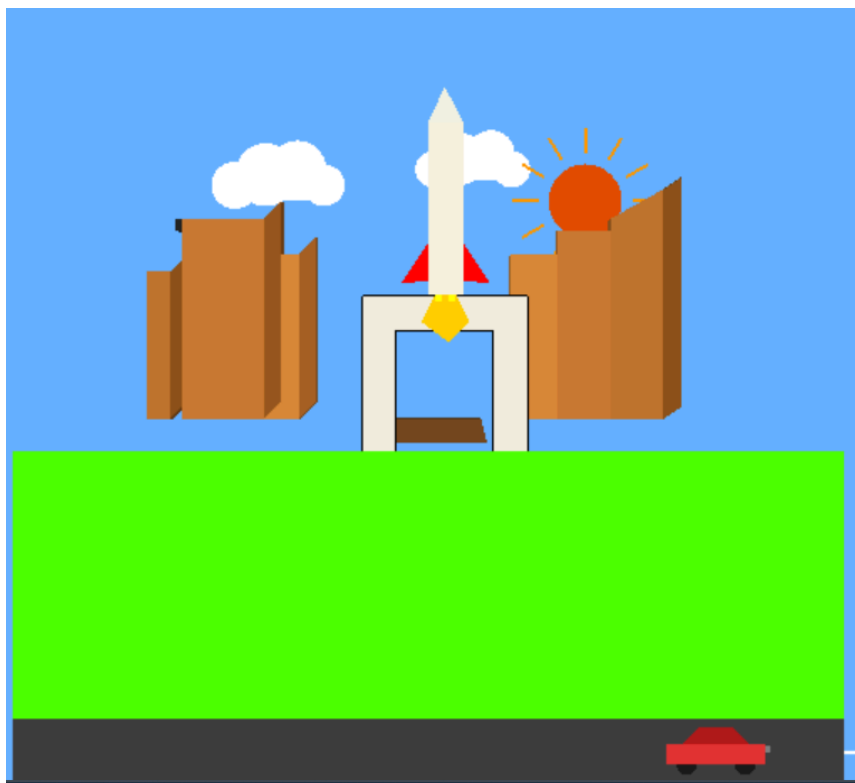
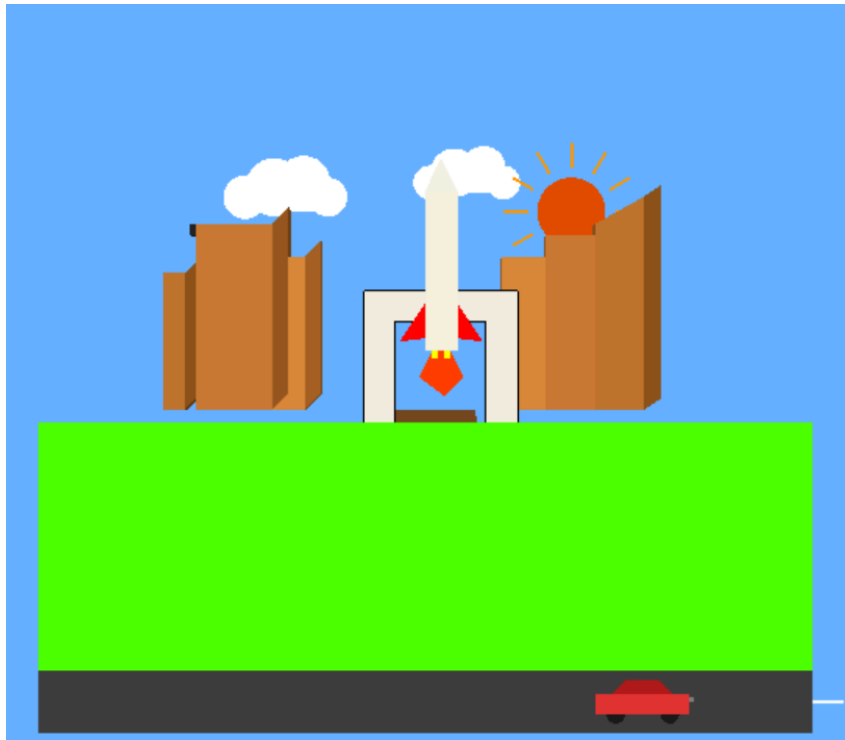


3d View angle (press C)

Night view



Camera angle :



Actual Sketch Design :

